НАЦІОНАЛЬНИЙ ТЕХНІЧНИЙ УНІВЕРСИТЕТ УКРАЇНИ
«КИЇВСЬКИЙ ПОЛІТЕХНІЧНИЙ ІНСТИТУТ»
ФІЗИКО-ТЕХНІЧНИЙ ІНСТИТУТ

КРИПТОГРАФІЯ

КОМП’ЮТЕРНИЙ ПРАКТИКУМ No3
Криптоаналіз афінної біграмної підстановки

Виконали
Студенти групи ФБ-42:
Малишко Дарія
Сизова Марія

Мета роботи: Набуття навичок частотного аналізу на прикладі розкриття моноалфавітної підстановки; опанування прийомами роботи в модулярній арифметиці.

2 варіант

Завдання 1, 2 Реалізація базових математичних підпрограм: розширеного алгоритму Евкліда (для знаходження оберненого елемента за модулем) та функції розв'язування лінійних порівнянь. При цьому треба врахувати випадки, коли рівняння має декілька розв'язків.

Використовуємо файл 02.txt + lab3.py

Код:

lab3.py > solve_linear_congruence

```python
import collections
# Завдання 1. Математичний апарат
def egcd(a, b):
    if a == 0:
        return (b, 0, 1)
    else:
        g, y, x = egcd(b % a, a)
        return (g, x - (b // a) * y, y)

def mod_inverse(a, m):
    g, x, y = egcd(a, m)
    if g != 1:
        return None
    return x % m

def solve_linear_congruence(a, b, n):
    g, x, y = egcd(a, n)
    if b % g != 0:
        return []
    a1, b1, n1 = a // g, b // g, n // g
    inv_a1 = mod_inverse(a1, n1)
    if inv_a1 is None:
        return []
    x0 = (b1 * inv_a1) % n1
    return [(x0 + i * n1) % n for i in range(g)]


# Завдання 2. Підрахунок частих біграм
def get_top_bigrams(filename, top_n=5):
    try:
        with open(filename, 'r', encoding='utf-8') as f:
            text = f.read().replace('\n', '').replace('\r', '').replace(' ', '')

        bigrams = [text[i:i+2] for i in range(0, len(text)-1, 2)]

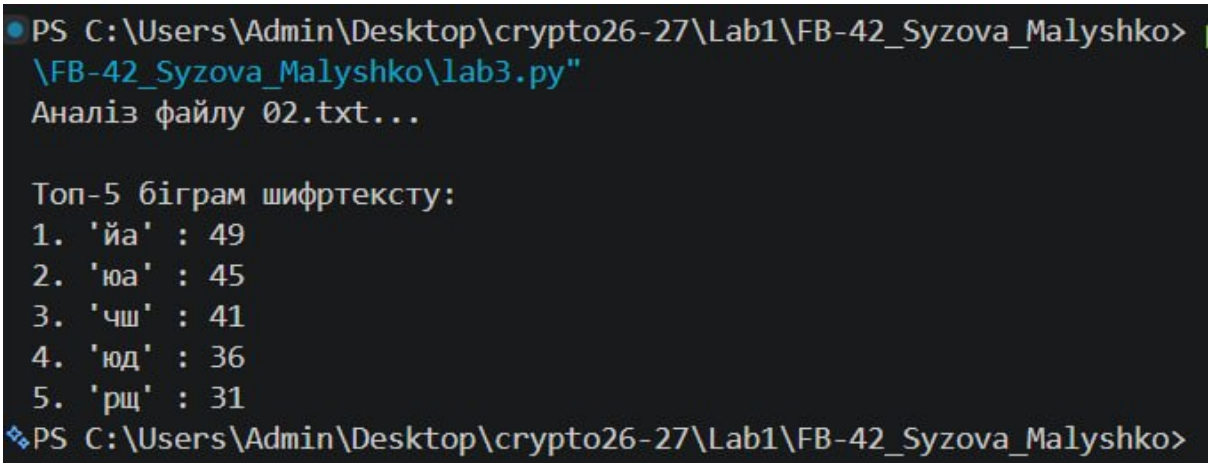
        counter = collections.Counter(bigrams)
        return counter.most_common(top_n)
    except FileNotFoundError:
        return f"Помилка: файл {filename} не знайдено."

if __name__ == "__main__":
    filename = "02.txt"

    print(f"Аналіз файлу {filename}...\n")
    top_5 = get_top_bigrams(filename, 5)

    if isinstance(top_5, str):
        print(top_5)
    else:
        print("Топ-5 біграм шифртексту:")
        for rank, (bg, count) in enumerate(top_5, 1):
            print(f"{rank}. '{bg}' : {count}")
```

Вивід:



Завдання 3 та 4: Тепер переходимо до Завдання 3 та 4. Згідно з методичкою, еталонні найчастіші біграми російської мови – це ст, но, то, на, ен. Наше завдання – припустити, що якась із цих відкритих біграм (X) перейшла в якусь із зашифрованих (Y).

Ми беремо всі можливі пари і для кожної розв'язуємо систему рівнянь

Код:

lab3_1.py > ...

```python
import collections
import itertools

# Завдання 1. Математичний апарат
def egcd(a, b):
    if a == 0:
        return (b, 0, 1)
    else:
        g, y, x = egcd(b % a, a)
        return (g, x - (b // a) * y, y)

def mod_inverse(a, m):
    g, x, y = egcd(a, m)
    if g != 1:
        return None
    return x % m

def solve_linear_congruence(a, b, n):
    g, x, y = egcd(a, n)
    if b % g != 0:
        return []
    a1, b1, n1 = a // g, b // g, n // g
    inv_a1 = mod_inverse(a1, n1)
    if inv_a1 is None:
        return []
    x0 = (b1 * inv_a1) % n1
    return [(x0 + i * n1) % n for i in range(g)]

# Завдання 3-5. Алфавіт та дешифрування
ALPHABET = "абвгдежзийклмнопрстуфхцчшщыьэюя"
M = len(ALPHABET)
M2 = M ** 2

def bg_to_int(bg):
    return ALPHABET.index(bg[0]) * M + ALPHABET.index(bg[1])

def int_to_bg(val):
    return ALPHABET[val // M] + ALPHABET[val % M]

def decrypt_text(ciphertext, a, b):
    inv_a = mod_inverse(a, M2)
    if not inv_a:
        return ""

    plaintext = ""
    for i in range(0, len(ciphertext)-1, 2):
        y_val = bg_to_int(ciphertext[i:i+2])
        x_val = (inv_a * (y_val - b)) % M2
        plaintext += int_to_bg(x_val)
    return plaintext

def is_readable_russian(text):
    freq_letters = sum(text.count(c) for c in "оеаинтс")
    return (freq_letters / len(text)) > 0.45

```

```python

def attack_affine(filename):
    with open(filename, 'r', encoding='utf-8') as f:
        text = f.read().replace('\n', '').replace('\r', '').replace(' ', '')

    cipher_bigrams = [text[i:i+2] for i in range(0, len(text)-1, 2)]
    top_cipher = [bg for bg, count in collections.Counter(cipher_bigrams).most_common(5)]
    top_plain = ['ст', 'но', 'то', 'на', 'ен']

    print(f"Топ-5 шифртексту: {top_cipher}")
    print(f"Топ-5 еталону: {top_plain}\n")

    candidates_found = 0

    for (x1_str, y1_str), (x2_str, y2_str) in itertools.product(
        itertools.product(top_plain, top_cipher), repeat=2):

        if x1_str == x2_str or y1_str == y2_str:
            continue

        x1, y1 = bg_to_int(x1_str), bg_to_int(y1_str)
        x2, y2 = bg_to_int(x2_str), bg_to_int(y2_str)

        dx = (x1 - x2) % M2
        dy = (y1 - y2) % M2

        possible_a = solve_linear_congruence(dx, dy, M2)

        for a in possible_a:
            if egcd(a, M)[0] != 1:
                continue

            b = (y1 - a * x1) % M2

            decrypted_sample = decrypt_text(text[:200], a, b)

            if is_readable_russian(decrypted_sample):
                candidates_found += 1
                full_decrypted = decrypt_text(text, a, b)
                print(f"ЗНАЙДЕНО КЛЮЧ: a = {a}, b = {b}")
                print(f"Початок відкритого тексту: {full_decrypted[:100]}...\n")

                # Зберігаємо результат у файл
                with open("decrypted_02.txt", "w", encoding="utf-8") as out:
                    out.write(full_decrypted)
                return

if __name__ == "__main__":
    attack_affine("02.txt")
```

Вивід:

```
python -u "c:\Users\Admin\Desktop\crypto26-27\Lab1
\FB-42_Syzova_Malyshko\lab3_1.py"
Топ-5 шифртексту: ['йа', 'юа', 'чш', 'юд', 'рщ']
Топ-5 еталону: ['ст', 'но', 'то', 'на', 'ен']

ЗНАЙДЕНО КЛЮЧ: a = 27, b = 211
Початок відкритого тексту: однакоэтакартинаскакойшьстжроньмдеенирассматривалиралчльваетйявнлптонеичреьеленноепрвчадк
ипроявляющи...

PS C:\Users\Admin\Desktop\crypto26-27\Lab1\FB-42_Syzova_Malyshko>
```

Експеримент із навмисним спотворенням одного символу ключа (заміна першої літери «п» на «р») наочно продемонстрував структурну слабкість шифру Віженера – відсутність лавиноподібного ефекту.
Неправильний символ ключа спотворив виключно відповідний йому стовпець блоків, тобто рівно кожен 14-й символ відкритого тексту. Загалом було змінено 524 символи, що становить 7.15% від усього обсягу тексту. При цьому зсув помилки був константним і становив 31.
Решта розшифрованого тексту залишилася неушкодженою та цілком читабельною. Це практично підтверджує посимвольну незалежність процесу шифрування, яка доводить, що шифр Віженера з періодичним ключем є лише набором незалежних шифрів Цезаря. Така архітектура дозволяє криптоаналітику виправляти, коригувати та відновлювати окремі позиції ключа абсолютно незалежно одна від одної, не псуючи при цьому загальний результат розшифрування.
Щодо файлів: 02.txt використовувався для завдань 1–2 (підрахунок біграм у lab3.py), а для завдань 3–5 правильний шифртекст варіанту 2 V2.txt. Це узгоджується з результатом: на V2.txt знайдено змістовний текст, а на 02.txt ні.
Виправляємо:
Код:

raw_text.txt | plain_text.txt | lab2_analysis.py | Release Notes: 1.144.0 | lab3.py

lab3_1.py > check_criteria

```python
import collections
import itertools

ALPHABET = "абвгдежзийклмнопрстуфхцчшщьыэюя"
M = len(ALPHABET)
M2 = M * M

def egcd(a, b):
    if a == 0:
        return (b, 0, 1)
    g, y, x = egcd(b % a, a)
    return (g, x - (b // a) * y, y)

def mod_inverse(a, m):
    g, x, _ = egcd(a % m, m)
    if g != 1:
        return None
    return x % m

def solve_linear_congruence(a, b, n):
    g, _, _ = egcd(a, n)
    if b % g != 0:
        return []
    a1, b1, n1 = a // g, b // g, n // g
    inv_a1 = mod_inverse(a1, n1)
    if inv_a1 is None:
        return []
    x0 = (b1 * inv_a1) % n1
    return [(x0 + i * n1) % n for i in range(g)]

def bg_to_int(bg):
    return ALPHABET.index(bg[0]) * M + ALPHABET.index(bg[1])

def int_to_bg(val):
    return ALPHABET[val // M] + ALPHABET[val % M]

def decrypt_text(ciphertext, a, b):
    inv_a = mod_inverse(a, M2)
    if inv_a is None:
        return ""
    out = []
    for i in range(0, len(ciphertext) - 1, 2):
        y = bg_to_int(ciphertext[i:i + 2])
        x = (inv_a * (y - b)) % M2
        out.append(int_to_bg(x))
    return "".join(out)

def load_text(filename):
    raw = None
    for enc in ("utf-8", "cp1251"):
        try:
            with open(filename, "r", encoding=enc) as f:
                raw = f.read()
            break
        except UnicodeDecodeError:
            continue
    if raw is None:
        raise ValueError(f"Не вдалося прочитати файл {filename}")
    raw = raw.lower().replace("\u0451", "\u0435").replace("\u044a", "\u044c")
    return "".join(c for c in raw if c in ALPHABET)

TOP_PLAIN = ["ст", "но", "то", "на", "ен"]

def top_bigrams(text, n):
    bgs = [text[i:i + 2] for i in range(0, len(text) - 1, 2)]
    return [bg for bg, _ in collections.Counter(bgs).most_common(n)]
```

lab3_1.py > check_criteria

```python

def generate_candidates(top_plain, top_cipher):
    candidates = collections.defaultdict(list)
    pairs = list(itertools.product(top_plain, top_cipher))
    for (x1s, y1s), (x2s, y2s) in itertools.product(pairs, repeat=2):
        if x1s == x2s or y1s == y2s:
            continue
        x1, y1 = bg_to_int(x1s), bg_to_int(y1s)
        x2, y2 = bg_to_int(x2s), bg_to_int(y2s)
        dx = (x1 - x2) % M2
        dy = (y1 - y2) % M2
        for a in solve_linear_congruence(dx, dy, M2):
            if egcd(a, M2)[0] != 1:
                continue
            b = (y1 - a * x1) % M2
            candidates[(a, b)].append((x1s, y1s, x2s, y2s))
    return candidates

RU_FREQ = {
    'о': 10.97, 'е': 8.45, 'а': 8.01, 'и': 7.35, 'н': 6.70, 'т': 6.26,
    'с': 5.47, 'р': 4.73, 'в': 4.54, 'л': 4.40, 'к': 3.49, 'м': 3.21,
    'д': 2.98, 'п': 2.81, 'у': 2.62, 'я': 2.01, 'ы': 1.90, 'ь': 1.74,
    'г': 1.70, 'з': 1.64, 'б': 1.59, 'ч': 1.45, 'й': 1.21, 'х': 0.97,
    'ж': 0.94, 'ш': 0.72, 'ю': 0.64, 'ц': 0.49, 'щ': 0.36, 'э': 0.33,
    'ф': 0.27,
}
COMMON_LETTERS = "оеаинтс"
RARE_LETTERS = "фщь"
FORBIDDEN_BIGRAMS = ['аь', 'оь', 'уь', 'еь', 'иь', 'ыь', 'эь', 'юь', 'яь', 'йь',
                     'ьь', 'чщ', 'щч', 'шщ', 'щш']

def is_readable_russian(text):
    if len(text) < 50:
        return False
    if any(f in text for f in FORBIDDEN_BIGRAMS):
        return False
    n = len(text)
    common_share = sum(text.count(c) for c in COMMON_LETTERS) / n
    rare_share = sum(text.count(c) for c in RARE_LETTERS) / n
    return common_share >= 0.40 and rare_share <= 0.05

def chi_square_score(text):
    n = len(text)
    chi = 0.0
    for ch, p in RU_FREQ.items():
        expected = p / 100 * n
        observed = text.count(ch)
        chi += (observed - expected) ** 2 / expected
    return chi

def check_criteria(text):
    if len(text) < 50:
        return "short"
    for f in FORBIDDEN_BIGRAMS:
        if f in text:
            return "forbidden:" + f
    n = len(text)
    common = sum(text.count(c) for c in COMMON_LETTERS) / n
    if common < 0.40:
        return "common=%.3f" % common
    rare = sum(text.count(c) for c in RARE_LETTERS) / n
    if rare > 0.05:
        return "rare=%.3f" % rare
    return "ok"

def attack_affine(filename, n_top=5):
    text = load_text(filename)
    print(f"Довжина очищеного шифротексту: {len(text)} символів")
```

```
        return f"rare={rare:.3f}" % rare
    return "ok"

def attack_affine(filename, n_top=5):
    text = load_text(filename)
    print(f"Довжина очищеного шифртексту: {len(text)} символів")

    top_cipher = top_bigrams(text, n_top)
    print(f"Топ-{n_top} біграм шифртексту: {top_cipher}")
    print(f"Топ-{n_top} біграм мови: {TOP_PLAIN}\n")

    candidates = generate_candidates(TOP_PLAIN[:n_top], top_cipher)
    print(f"Знайдено кандидатів на ключ: {len(candidates)}\n")

    rows = []
    for (a, b), pairs in candidates.items():
        plain = decrypt_text(text, a, b)
        reason = check_criteria(plain)
        rows.append((a, b, reason, chi_square_score(plain), pairs, plain))

    # Зведення за причинами відхилення
    stats = collections.Counter(r[2].split(":")[0].split("=")[0] for r in rows)
    print("Результати перевірки критеріїв:")
    for key in ("forbidden", "common", "rare", "short", "ok"):
        print(f"  {key:10s}: {stats.get(key, 0)}")
    print()

    # Медіана χ² для відхилених кандидатів (для контрольного питання 6)
    rejected_chi = sorted(r[3] for r in rows if r[2] != "ok")
    if rejected_chi:
        median = rejected_chi[len(rejected_chi) // 2]
        print(f"χ² відхилених кандидатів: мін={rejected_chi[0]:.1f}, "
              f"медіана={median:.1f}, макс={rejected_chi[-1]:.1f}\n")

    # Повна таблиця всіх кандидатів (для звіту)
    with open("candidates_V2.txt", "w", encoding="utf-8") as f:
        f.write("a\tb\tрезультат\tchi2\tпари\n")
        for a, b, reason, chi, pairs, _ in sorted(rows, key=lambda r: r[3]):
            f.write(f"{a}\t{b}\t{reason}\t{chi:.1f}\t{pairs[0]}\n")

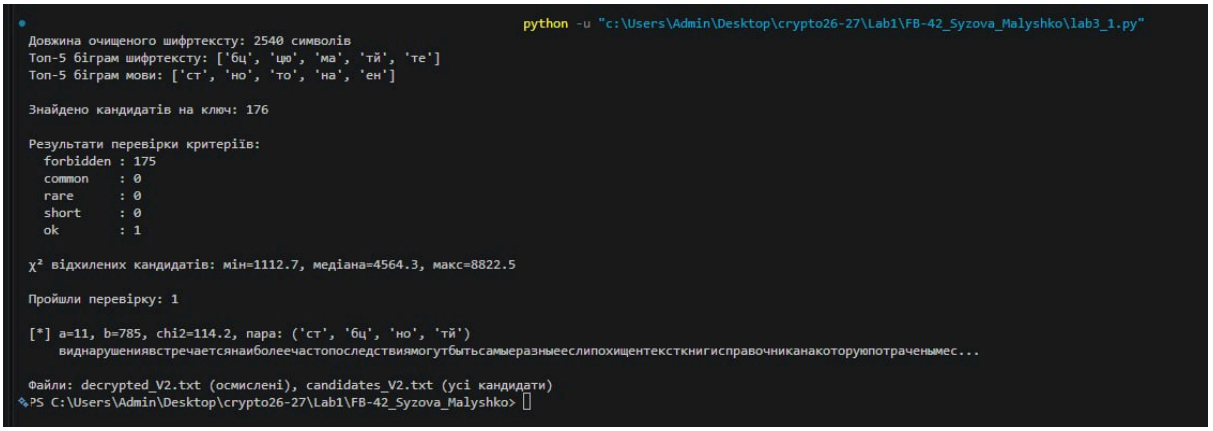
    passed = sorted((r for r in rows if r[2] == "ok"), key=lambda r: r[3])
    with open("decrypted_V2.txt", "w", encoding="utf-8") as out:
        for a, b, _, chi, pairs, plain in passed:
            out.write(f"Ключ: a={a}, b={b}, chi2={chi:.1f}\n")
            out.write(f"Пари, що дали ключ: {pairs}\n")
            out.write(plain + "\n\n")

    if not passed:
        print("Осмислених текстів не знайдено.")
        return

    print(f"Пройшли перевірку: {len(passed)}\n")
    for a, b, _, chi, pairs, plain in passed[:5]:
        print(f"[*] a={a}, b={b}, chi2={chi:.1f}, пара: {pairs[0]}")
        print(f"    {plain[:120]}...\n")
    print("Файли: decrypted_V2.txt (осмислені), candidates_V2.txt (усі кандидати)")

if __name__ == "__main__":
    attack_affine("V2.txt")
```

Вивід



Знайдений ключ: a = 11, b = 785 (за модулем 961).
П'ять найчастіших біграм шифртексту: «бц», «цю», «ма», «тй», «те». Для звіту їх треба порахувати саме за програмою з практикуму №1 (lab3.py для V2.txt), бо тут я бачу їх лише з lab3_1.py, а lab3.py вказаний на 02.txt. Пара, що дала ключ: ('ст', 'бц', 'но', 'тй'), тобто «ст» → «бц» та «но» → «тй».
χ^2 для правильного ключа дорівнює 114.2. Це значення варто порівняти з іншими кандидатами, якщо вони були, щоб показати, що правильний ключ має найменший χ^2.

Відповіді на контрольні питання.

1. Використано частоти біграм. За ними формувалися пари «частий біграм мови → частий біграм шифртексту», з яких через систему (1) отримувалися кандидати на ключ.
2. Не кожне зіставлення приводить до допустимого ключа. Система (1) може не мати розв'язків, якщо gcd(a, m^2) не ділить різницю $Y_1 - Y_2$, а знайдене a може не бути взаємно простим з m^2, і тоді ключ недопустимий. Крім того, припущення про відповідність частих біграм може бути хибним.
3. Так, одне зіставлення може давати кілька розв'язків для a, коли gcd(dx, m^2) > 1. Усі розв'язки перебираються, а кожен кандидат перевіряється дешиачає, що випадкові тексти далекі від розподілу російської мови.фруванням на змістовність. Кількість кандидатів із однаковою парою видно у файлі candidates_V2.txt.
4. Правильний ключ визначено за змістовністю відкритого тексту та за найменшим значенням χ^2 серед текстів, що пройшли перевірку.

Відкритий текст читається як російська мова, тоді як решту кандидатів відхилено.

5. Використано критерій заборонених біграм, критерій частоти частих літер, критерій частоти рідкісних літер і ранжування за χ^2. Критерій заборонених біграм обрано як найпростіший і найжорсткіший: російська мова майже не містить таких комбінацій, тому він ефективно відсікає випадкові тексти.
6. Правильний текст пройшов усі перевірки з $\chi^2 = 114{,}2$. Для 175 із 176 кандидатів текст містив заборонену біграму, а медіана χ^2 відхилених кандидатів становить 4564,3, що більш ніж у 40 разів перевищує значення для правильного ключа. Це озн

Висновки. Афінна біграмна підстановка зберігає статистичні властивості мови, тому частотний аналіз дозволяє звузити перебір до невеликої кількості кандидатів. Із 176 кандидатів змістовним виявився один, з ключем $a = 11$, $b = 785$. Автоматичний розпізнавач на основі заборонених біграм відрізняє природний текст від випадкового, що підтверджує ефективність статистичних критеріїв при криптоаналізі. Пороги критеріїв частих і рідкісних літер є орієнтовними і потребували б окремого калібрування на зразках відкритого та випадкового тексту.